TALLER DE METODOLOGÍAS ÁGILES · 2026-2 · TERCER CORTE

Punto de control del proyecto final

Programar en pareja con una persona o con un agente: qué se gana, qué se pierde y qué aprendizaje desaparece

TEMA DE EXPOSICIÓN

3 · Pair programming

PRÁCTICA QUE ASISTE EL AGENTE

Desarrollo de una tarea con el agente como copiloto

CASO BASE

Notitas: un tablero kanban de notas adhesivas

PAREJA

Ana Sofía Arango Yanza · Juan José Arias Chacua

PROFESOR

César Jesús Pardo Calvache, Ph.D.

FECHA Y VERSIÓN

Entrega del caso base: martes 3 de noviembre de 2026 · v3 (31/10; reemplaza la v2, que usaba CAMEIA-lite)

REPOSITORIO

github.com/Sofii141/CAMEIA-lite · ramas `main` (código) y `documentacion`

0 Estado frente a lo que pide el punto de control

Con su aprobación, cambiamos el caso base de CAMEIA-lite a **Notitas**, y el plazo del caso base quedó para el martes 3/11. Todavía no hemos usado el agente.

CASO BASE

24 casos

10 S · 12 M · 2 L · **10 difíciles**

ESPECIFICACIÓN V1

1 agente

8 campos · OpenCode + DeepSeek V4.1 Flash

LÍNEA BASE

10 tareas

pareja humana, sin IA (§5)

Requisito del enunciado	Dónde está	Estado
Caso base con ≥ 10 casos que incluye material difícil	§2 · repo `main` (`TAREAS.md`)	Listo
Caso congelado antes de tocar el agente	tag `congelado-notitas-v1` en `main`	Antes de medir
Línea base medida con GQM, ≥ 10 casos y condiciones registradas	§3, §4, §5 y bitácora v3 (§11)	Hasta el 3/11
Especificación de agentes v1 con sus 8 campos	§7	Lista
Tres casos para entregar a otra pareja y tres que recibimos	`02 - Caso base/Casos entregados` y `Casos recibidos`	Según la pareja asignada

REGLA QUE SEGUIMOS

La línea base se mide **en vivo y grabada**, en el orden que fija el sorteo y antes de usar el agente. En §5 va **solo** lo que ya está medido: el script `resumen_medicion.py` lo copia desde la bitácora. Nada se reconstruye de memoria.

1 Tema, práctica e hipótesis

El enunciado asigna al tema 3 la práctica **"desarrollo de una tarea con el agente como copiloto"** y sugiere medir tiempo, retrabajo y errores detectados a tiempo. Para que el proyecto y la exposición cuenten una sola historia, agregamos la pregunta central del tema: **qué aprendizaje se pierde** cuando el compañero es un agente que no necesita entender.

Qué dice la literatura revisada por pares (y qué esperamos ver)

Hallazgo publicado	Hipótesis para nuestro caso
Hannay et al. 2009 (metaanálisis, *IST*) y **Arisholm et al. 2007** (*IEEE TSE*): en general, la pareja no reduce el tiempo, cuesta un 84 % más de esfuerzo y su efecto depende de la complejidad.	**H1:** con el agente, la duración baja en las tareas S y M, pero no en las difíciles.
Perry et al. 2023 (*ACM CCS*): con IA el código fue menos seguro y, aun así, quienes la usaron creían que era más seguro.	**H2:** con el agente tardamos más de lo que pronosticamos más veces que en pareja.
Welter et al. 2025 (*ASE*): las sugerencias de Copilot se aceptan con menos escrutinio que las de un compañero humano.	**H3:** con el agente se escapan más errores a las pruebas ocultas, sobre todo en la trampa de seguridad (t14) y en la de mutabilidad (t20).
Plonka et al. 2015 (*IJHCS*) y **Shihab et al. 2025** (*ICER*): en pareja se transfiere conocimiento; con Copilot, los estudiantes no entendían por qué funcionaban las sugerencias. Contrapeso: **Kazemitabaar et al. 2023** (*CHI*) no encontró menos retención.	**H4:** el codewalk baja con el agente, sobre todo el del compañero que no participó.

EL ESTUDIO MÁS PARECIDO AL NUESTRO

Fan et al. 2025 (*International Journal of STEM Education*) comparó pareja con IA, pareja humana y trabajo individual con 234 estudiantes. Con IA hubo mejor desempeño en las tareas, pero la pareja humana tuvo la mayor percepción de colaboración. Nosotros miramos lo que ese estudio no mide: tiempo, errores escapados, retrabajo y comprensión, tarea por tarea.

2 El caso base: Notitas

Un tablero kanban de notas adhesivas (*Por hacer · En curso · Hecho*). Se mide el desarrollo de las **24 reglas del backend**; el front ya está hecho y no se mide.

De dónde sale y por qué así

Lo diseñamos nosotros, como permite el enunciado ("construyanlo ustedes"), con su aprobación del 31/10. Reemplaza a CAMEIA-lite, que sigue en el repositorio (tag `congelado-v1`) y nunca se llegó a medir.

Cada tarea es una función o clase pura de Python con sus criterios de aceptación. Así el cronómetro mide la práctica y no la configuración, y las tareas se pueden comparar entre condiciones. Ninguno de los dos había programado estas reglas antes.

Qué lo hace representativo y difícil

- **10 S · 12 M · 2 L**, asignadas por sorteo estratificado (§4).
- **10 difíciles**: concurrencia, zona horaria, Unicode, inyección de HTML (XSS), mutabilidad, fusión con conflictos, ventana deslizante y reglas que parecen obvias pero no lo son.
- **41 pruebas visibles** (guían el desarrollo) y **72 ocultas** (miden los errores escapados).
- Las 113 pruebas fallan con los esqueletos y pasan con soluciones de referencia que no están en el repositorio (comprobado el 31/10).

Las 24 tareas

ID	Tarea	Tam.	Por qué es difícil
`t01`	Validar el título de una nota	S	
`t02`	Color de la nota	S	
`t03`	Transiciones entre columnas	M	Difícil Muchas combinaciones y una regla que parece obvia pero no lo es (no se salta "En curso")
`t04`	Límite WIP de "En curso"	M	Difícil Reordenar dentro de la misma columna no debe contar como una nota más
`t05`	Insertar en una posición	M	
`t06`	Filtrar por etiquetas	S	
`t07`	Buscar sin tildes ni mayúsculas	M	Difícil Unicode: tildes, ñ y letras como ß que no se resuelven con lower()
`t08`	Estado de vencimiento	M	Difícil Zona horaria: a las 10 p. m. de Bogotá ya es el día siguiente en UTC
`t09`	Normalizar etiquetas	S	
`t10`	Qué notas se archivan	S	

ID	Tarea	Tam.	Por qué es difícil
t11	Deshacer movimientos	M	
t12	Dos personas mueven la misma nota	L	Difícil Concurrencia: dos llamadas al mismo tiempo con la misma versión; solo una puede ganar
t13	Exportar e importar el tablero	M	
t14	Contenido seguro (sin inyectar HTML)	M	Difícil Trampa de seguridad: hay que escapar antes de dar formato, no después
t15	Resumen por columna	S	
t16	Ordenar por prioridad y fecha	S	
t17	Paginar el listado	S	
t18	Iniciales del responsable	S	
t19	Progreso de la checklist	M	
t20	Duplicar una nota	S	Difícil Trampa de mutabilidad: una copia superficial comparte las listas con la original
t21	Fusionar cambios de dos dispositivos	L	Difícil Fusión a tres bandas, empates y fechas con zonas horarias distintas
t22	Notas bloqueadas	M	
t23	Tiempo de ciclo	M	Difícil La nota puede ir y volver entre columnas, y los eventos pueden llegar desordenados
t24	Límite de notas por minuto	M	Difícil Ventana deslizante: los bordes y los intentos rechazados

LIMITACIONES DECLARADAS

- **Es un caso diseñado para el experimento**, no el backlog de un proyecto en curso: los resultados no se generalizan a un sistema real.
- Las tareas, las pruebas y la demo se generaron con Claude Code. Para que el caso no quede "a la medida" del agente, el agente de los ciclos es **otro modelo** (DeepSeek V4.1 Flash en OpenCode); además nadie ve las pruebas ocultas antes de terminar, el agente no tiene acceso a ellas y se incorporan 3 casos de otra pareja.
- Las tareas son reglas aisladas: no medimos el costo de integrarlas con el resto del sistema.

3 GQM

Objetivo: analizar el desarrollo de una tarea con un agente copiloto, frente al pair programming humano, para saber si cambian el tiempo, los errores que se escapan, el retrabajo y la comprensión del código, en las 24 tareas de Notitas.

Son cinco métricas **directas**: cada una es un número que se anota tal cual, sin fórmulas ni métricas compuestas. Las tres primeras son las que sugiere el enunciado para el tema 3.

Pregunta	Métrica	Cómo se toma	Qué NO representa	Respaldo
Q1 ¿Se termina más rápido?	**M1** Minutos hasta que pasan las pruebas visibles	Hora de inicio y de fin, menos pausas	Productividad general	Hannay 2009; Shihab 2025
Q2 ¿Se detectan los errores a tiempo?	**M2** Pruebas ocultas que fallan al declarar "terminado"	`pytest tests/ocultas -k tNN`	Defectos de diseño o de legibilidad	Arisholm 2007; Dakhel 2023
Q3 ¿Hay retrabajo?	**M3** Minutos de corrección después de "terminado"	Cronómetro aparte (0 si no hubo)	El tamaño de la corrección	Enunciado del curso
Q4 ¿Qué aprendizaje se pierde?	**M4** Codewalk 0–5, del autor y del compañero	5 preguntas; quien responde no mira el código	La habilidad general	Plonka 2015; Welter 2025
Q5 ¿Nos engaña la sensación de velocidad?	**M5** Pronóstico en minutos, anotado antes de empezar	Se compara con M1	El resultado real	Perry 2023; Lee 2025

En los ciclos, la bitácora registra además cada uso del agente con su decisión (aceptar, corregir o rechazar), su razón y su responsable.

Las 5 preguntas del codewalk (1 punto cada una, máximo 5 min por persona)

1. ¿Qué hace la función, en una frase?
2. ¿Qué caso borde maneja y dónde?
3. ¿Por qué se eligió esta forma y no otra?
4. Si mañana falla con la entrada X, ¿dónde mirarías primero? *(X la escoge el evaluador)*
5. ¿Qué cambiarías si el requisito cambia en Y? *(Y la escoge el evaluador)*

4 Diseño de la medición

	Línea base (hasta el 3/11)	Ciclo 1	Ciclo 2
Condición	Pareja humana: conductor y navegante, cambian cada 15 min, sin IA	Una persona + el agente copiloto; el compañero no participa	
Tareas	10 por sorteo: **5 S · 4 M · 1 L**	12 por sorteo (6 + 6): **5 S · 6 M · 1 L**; cada persona conduce 6. En el ciclo 2 se suman los 3 casos recibidos	
Evidencia	Grabación de pantalla, commits por tarea (`tNN: inicio / terminado / retrabajo`), hoja de medición, hoja de sesiones y, en los ciclos, una fila de bitácora por cada uso del agente		

Protocolo por tarea

1. Anotar el pronóstico de minutos (M5).
2. Iniciar el cronómetro al abrir el enunciado y hacer el commit `tNN: inicio` .
3. Trabajar solo con las pruebas visibles (`python -m pytest -k tNN` ; por defecto no corre las ocultas).
4. Declarar "terminado" cuando pasan, parar el cronómetro (M1) y hacer el commit `tNN: terminado` .
5. Correr las pruebas ocultas y anotar cuántas fallan (M2). Si se corrige, el tiempo de la corrección es retrabajo (M3), con el commit `tNN: retrabajo` .
6. Codewalk al autor y al compañero (M4).
7. Llenar la hoja de medición en ese momento, no al final del día.

CONTROLES PARA QUE LA COMPARACIÓN VALGA

- **Sorteo reproducible y estratificado** (`python sorteo.py <semilla>`): cupos fijos por tamaño, las difíciles repartidas alternando entre condiciones y el orden de las tareas también sorteado. La semilla y la salida quedan en `registro/` .
- **Caso congelado** (tag `congelado-notitas-v1`) antes del sorteo y de la primera medición.
- Si alguien abre las **pruebas ocultas antes de tiempo**, la tarea se descarta con la marca `[C]` y se usa una de repuesto (2 M).
- En los ciclos, el agente trabaja en una copia del repositorio **sin** `tests/ocultas` .

5 Línea base

Se mide en 2 o 3 sesiones antes del 3/11, en el orden del sorteo. Cada sesión queda en la hoja *Sesiones* de la bitácora (quién, cuándo, cuánto duró, herramientas, interrupciones y grabación). Abajo está lo medido hasta la fecha de este documento.

PENDIENTE DE MEDIR

Esta sección la llena `07 - Analisis/Scripts/resumen_medicion.py` a partir de la bitácora v3, después de cada sesión. No se escribe a mano.

6 Cómo presentaremos los resultados

Simple y suficiente: con pocas tareas por condición, mostramos todos los datos y decimos qué no demuestran.

Qué reportamos	Ejemplo	Fuente
Mediana [mínimo–máximo] de M1 y M4, por tamaño	"Tareas M: 32 min [21–48] en pareja y 25 min [18–60] con el agente"	Kitchenham et al. 2017, *EMSE*
Conteos de M2 y M3	"Se escaparon 4 de 30 errores; 3 de 10 tareas tuvieron retrabajo"	Arisholm et al. 2007
M1 frente a M5	"En 7 de 12 tareas tardamos más de lo que creíamos"	Perry et al. 2023
Decisiones de la bitácora	"De 48 usos: 20 aceptados, 21 corregidos y 7 rechazados"	Enunciado del curso
Sin p-valores ni causalidad; en cada cifra, qué **no** demuestra	"No prueba que el agente cause la diferencia: n ≈ 10, tareas distintas en cada condición"	Kitchenham et al. 2002, *IEEE TSE*; Wohlin et al. 2012
Recálculo independiente de al menos una mediana	Script, hoja de cálculo y la otra persona a mano	—

7 Especificación del agente · v1

Campo	Contenido
Rol y alcance	Hace de navegante en el desarrollo de *una* tarea de Notitas: propone un enfoque, el código y los casos borde que conviene probar. La persona conduce y decide.
Entradas	Solo el enunciado y los criterios de aceptación de la tarea (`TAREAS.md`), su archivo `src/notitas/tNN.py` y sus pruebas visibles.
Salidas	En markdown fijo: un plan de 3 a 6 pasos, el diff (solo en `tNN.py`), los supuestos que hizo y los casos borde que cree que faltan, como texto.
Herramientas y permisos	**OpenCode** con el modelo gratuito **DeepSeek V4.1 Flash**, el mismo para los dos durante todo el ciclo (modelo y versión se anotan en el primer uso). Trabaja en una copia del repositorio **sin** `tests/ocultas` , con un `opencode.json` que le niega leer las ocultas, editar las pruebas, hacer commit o push, salir de la carpeta y buscar en internet, y que lo obliga a pedir permiso para todo lo demás. No es de la familia Claude, con la que se construyó el caso.
Prohibiciones	No hacer commit, push ni merge. No modificar `tests/` ni otros archivos. No leer las pruebas ocultas. No desactivar ni saltarse pruebas. No inventar requisitos. No manejar credenciales ni datos personales.
Criterio de validación	Se acepta solo si pasan las pruebas visibles, cumple cada criterio de aceptación y quien conduce puede explicar cada línea sin mirar al agente.
Punto de control humano	Antes de cada commit, quien conduce decide aceptar, corregir o rechazar y lo registra en la bitácora con su razón. Las pruebas ocultas las corre la persona. Después viene el codewalk con el compañero.
Versión	v1 · 31/10/2026 (la misma v1 del 29/10, con las rutas de Notitas). Cada cambio da una versión nueva, con el dato de la bitácora que la motivó.

Empezamos con un solo agente. Solo agregaremos un segundo ("revisor") en el ciclo 2 si la bitácora del ciclo 1 muestra un problema que el primero no resuelve.

8 Calendario

31 oct
Caso Notitas aprobado y construido

1–3 nov
Revisar, congelar, sortear y medir la línea base

Ciclo 1
Con el agente, bitácora y spec v2

Ciclo 2
3 casos recibidos, 3 fallos y video

16–18 nov
Reporte, diapositivas y exposición

9 Preguntas para el profesor

Solo las que pueden cambiar el diseño o el calendario. Su respuesta la guardamos por escrito en `09 - Control de entrega`.

☐ **Fechas de los ciclos.** Con el plazo del caso base corrido al 3/11, ¿mantenemos el ciclo 1 del 2 al 6 de noviembre (empezándolo al cerrar la línea base) o se corre también?

☐ **Intercambio.** ¿Con qué pareja intercambiamos los 3 casos, y en qué formato? Proponemos una ficha con enunciado, criterios de aceptación y pruebas, como las de Notitas.

10 Fuentes

DOI verificados en Crossref y OpenAlex el 29/10/2026. Lista completa, con base de datos y estado de lectura: `01 - Exposicion/Fuentes/29102026_01_Fuentes_v2.md`.

Revisadas por pares

- Arisholm, E., Gallis, H., Dybå, T., & Sjøberg, D. I. K. (2007). Evaluating pair programming with respect to system complexity and programmer expertise. *IEEE Transactions on Software Engineering, 33*(2), 65–86. doi:10.1109/TSE.2007.17
- Fan, G., Liu, D., Zhang, R., & Pan, L. (2025). The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance. *International Journal of STEM Education, 12*, 16. doi:10.1186/s40594-025-00537-3
- Hannay, J. E., Dybå, T., Arisholm, E., & Sjøberg, D. I. K. (2009). The effectiveness of pair programming: A meta-analysis. *Information and Software Technology, 51*(7), 1110–1122. doi:10.1016/j.infsof.2009.02.001
- Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. *CHI 2023*, 1–23. doi:10.1145/3544548.3580919
- Kitchenham, B. et al. (2002). Preliminary guidelines for empirical research in software engineering. *IEEE TSE, 28*(8), 721–734. doi:10.1109/TSE.2002.1027796
- Kitchenham, B., Madeyski, L., Budgen, D., Keung, J., Brereton, P., Charters, S., Gibbs, S., & Pohthong, A. (2017). Robust statistical methods for empirical software engineering. *Empirical Software Engineering, 22*(2), 579–630. doi:10.1007/s10664-016-9437-5
- Ko, A. J., LaToza, T. D., & Burnett, M. M. (2015). A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering, 20*(1), 110–141. doi:10.1007/s10664-013-9279-3
- Lee, H.-P. et al. (2025). The impact of generative AI on critical thinking. *CHI 2025*, 1–22. doi:10.1145/3706598.3713778
- Moradi Dakhel, A. et al. (2023). GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software, 203*, 111734. doi:10.1016/j.jss.2023.111734
- Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? *ACM CCS 2023*, 2785–2799. doi:10.1145/3576915.3623157
- Plonka, L., Sharp, H., van der Linden, J., & Dittrich, Y. (2015). Knowledge transfer in pair programming: An in-depth analysis. *International Journal of Human-Computer Studies, 73*, 66–78. doi:10.1016/j.ijhcs.2014.09.001
- Shihab, M. I. H., Hundhausen, C., Tariq, A., Haque, S., Qiao, Y., & Mulanda, B. (2025). The effects of GitHub Copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks. *ICER 2025*, 407–420. doi:10.1145/3702652.3744219
- Vargha, A., & Delaney, H. D. (2000). A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics, 25*(2), 101–132. doi:10.3102/10769986025002101
- Welter, A., Schneider, N., Dick, T., Weis, K., Tinnes, C., & Wyrich, M. (2025). An empirical study of knowledge transfer in AI pair programming. *ASE 2025*, 166–177. doi:10.1109/ASE63991.2025.00022
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. doi:10.1007/978-3-642-29044-2

- Basili, V., Caldiera, G., & Rombach, H. D. (1994). The Goal Question Metric approach. En *Encyclopedia of Software Engineering*. Wiley.

Literatura gris (no revisada por pares; solo como contexto)

- Becker, J., Rush, N., Barnes, B., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. METR, preprint arXiv:2507.09089.

11 Anexos y enlaces

No adjuntamos archivos aparte: todo está en el repositorio público github.com/Sofii141/CAMEIA-lite, con su historial de versiones.

Anexo	Enlace
Código del caso base Notitas (24 tareas, pruebas, sorteo, demo)	rama main · TAREAS.md
Resultado del sorteo y registro de sesiones	registro/
Bitácora y medición (hoja de cálculo)	31102026_01_Bitacora_v3.xlsx
Especificación del agente v1 (con historial)	31102026_01_Spec_Copiloto_v1.md
GQM v3	31102026_01_GQM_v3.md
Fuentes (revisadas por pares, con DOI)	29102026_01_Fuentes_v2.md
Caso anterior (CAMEIA-lite) y registro de cambios	tag congelado-v1 · CHANGELOG.md

12 Declaración de uso de IA

Herramienta	Para qué	Qué verificamos nosotros
Claude Code	Leer los enunciados; proponer la estructura; generar las 24 tareas de Notitas con sus pruebas y soluciones de referencia, el backend y el front de la demo; buscar fuentes y verificar sus DOI en Crossref y OpenAlex; revisar la coherencia de los documentos con un segundo agente revisor; maquetar este documento.	**Completar antes de entregar**: que revisamos las 24 tareas antes de congelar y qué fuentes leímos (las que tengan `[V]`).

Ningún dato de línea base ni de bitácora fue generado por IA: esos datos salen solo de nuestras sesiones cronometradas y grabadas.